Sí. Revisando lo que definimos y comparándolo con lo que realmente parece que querés construir, **haría una corrección importante respecto de mi respuesta anterior**:

> ***Git no debería quedar limitado a `Programming/`. El repositorio Git debería abarcar todo `Trading Lab/`.***

Eso nos da una arquitectura bastante más coherente: el laboratorio completo es versionable, mientras que Git sigue siendo especialmente importante para el código.

Además, la documentación actual de Codex encaja muy bien con esta idea: `AGENTS.md` se descubre jerárquicamente desde el root hacia el directorio de trabajo, y OpenAI está usando `AGENTS.md` + `.agents/skills/` para convertir reglas y workflows en parte del repositorio. También existe una guía explícita de migración desde `CLAUDE.md`/`.claude/skills` hacia `AGENTS.md`/`.agents/skills`.
(https://developers.openai.com/cookbook/examples/gpt-5/codex_prompting_guide?trk=article-ssr-frontend-pulse_little-text-block&utm_source=chatgpt.com)

1. La arquitectura que ahora considero más correcta

Yo la dejaría así:

```
Trading Lab/
│
├── README.md
├── AGENTS.md
├── .gitignore
├── .agents/
│   └── skills/
│
├── Session Reports/
│
├── Documentation/
│
├── Programming/
│   ├── Applications/
│   ├── Automation/
│   ├── Scripts/
│   ├── Database/
│   │   ├── Schema/
│   │   ├── Migrations/
│   │   ├── Queries/
│   │   └── Seeds/
│   └── Integrations/
│
└── Projects/
    │
    ├── Scalping 9 Robots/
    │   ├── README.md
    │   ├── AGENTS.md
    │   ├── MT5/
    │   ├── SQX/
    │   ├── Documentation/
    │   └── Session Reports/
    │
    └── XAUUSD H4 Solo Compras/
        ├── README.md
        ├── AGENTS.md
        ├── MT5/
        ├── SQX/
        ├── Documentation/
        └── Session Reports/
```

No agregaría ahora una carpeta `Data/` ni `Research/` ni `Experiments/` a nivel superior.

La razón es que tus cuatro directorios tienen una lógica muy buena:

Directorio	**Función**
`Session Reports`	memoria cronológica del laboratorio
`Documentation`	conocimiento e investigación
`Programming`	infraestructura y software
`Projects`	experimentos/conjuntos de estrategias concretos

Y eso es mucho más estable que intentar convertir cada función conceptual del trading en una carpeta.

2. La gran separación que tenemos que mantener

Creo que ésta es la clave de todo el sistema:

El Trading Lab tiene cuatro tipos de información

1. Conocimiento

```
Documentation/
```

2. Historia

```
Session Reports/
```

3. Infraestructura

```
Programming/
```

4. Trabajo experimental

```
Projects/
```

Y dentro de cada proyecto:

```
SQX → generación/investigación de estrategias
MT5 → implementación/ejecución/testing
Documentation → conocimiento específico del proyecto
Session Reports → historia del proyecto
```

Esto me parece mucho más sólido que hacer carpetas como `Research`, `Strategy Generation`, `Validation`, etc. porque esos últimos son **procesos**, no necesariamente categorías de almacenamiento.

---

3. ¿Dónde entra Git?

Acá cambiaría bastante lo que dije antes.

Git en el root

```
Trading Lab/
└── .git/
```

No:

```
Programming/
└── .git/
```

Git debería versionar **todo aquello que constituye el conocimiento y la definición reproducible del laboratorio**.

Por ejemplo:

Sí versionaría

```
AGENTS.md
README.md

Documentation/**/*.md
Documentation/**/*.pdf   ← según tamaño/licencia

Programming/**

Projects/**/README.md
Projects/**/Documentation/**
Projects/**/Session Reports/**
Projects/**/config/**
Projects/**/source code
.agents/skills/**
```

No versionaría normalmente

```
*.env
credentials
API keys
bases de datos físicas
logs temporales
binarios generados
*.ex5
*.exe
datasets gigantes
resultados duplicados/generados automáticamente
```

Y los reportes MT5/SQX hay que decidirlos individualmente. Si son pequeños y forman parte del registro experimental, pueden permanecer como artefactos del proyecto; si empiezan a ser miles de archivos o muy pesados, los sacamos del tracking y dejamos su referencia/hashes en el sistema.

---

4. Una distinción que considero fundamental: Git ≠ experimentos

Esto es probablemente lo más importante de toda la arquitectura.

No usaría Git para representar cada experimento.

Por ejemplo, no haría:

```
branch:
experiment-001
experiment-002
experiment-003
...
```

Eso terminaría siendo inmanejable.

Git responde:

> ***¿Cómo cambió nuestro software, configuración y documentación?***

La base de datos responde:

> *¿Qué experimento hicimos y qué resultado obtuvimos?*

Y el Session Report responde:

> *¿Qué hicimos durante esta sesión y qué decidimos?*

Son tres cosas distintas.

---

5. Incluso podemos conectar Git con los experimentos

Acá aparece algo muy potente.

Cada experimento podría guardar:

```
experiment_id
project_id
strategy_version
git_commit
SQX_version
MT5_version
data_version
parameters
results
```

Por ejemplo:

```
Experiment: EXP-2026-00421

Project:
Scalping 9 Robots

Strategy:
SR-017

Git commit:
8f31c2a

Engine:
MT5

Symbol:
XAUUSD

Timeframe:
M5

Period:
2022-01-01 → 2025-12-31

Result:
Profit Factor = 1.47
Drawdown = 12.3%
Trades = 1832
```

Entonces dentro de dos años podemos preguntar:

> *"¿Con qué código exacto se obtuvo este resultado?"*

Y tener una respuesta inequívoca.

Eso es **reproducibilidad experimental**, que para el tipo de Trading Lab que estás planteando es muchísimo más importante que simplemente guardar PDFs de reportes.

---

6. ¿Una base de datos por proyecto?

Acá mi recomendación es bastante clara:

No.

No haría:

```
DB Scalping 9 Robots
DB XAUUSD H4
DB EURUSD Mean Reversion
DB ...
```

Tendríamos una base de datos central:

```
Trading Lab DB
```

y cada proyecto tendría su `project_id`.

Conceptualmente:

```
Trading Lab DB
│
├── Projects
│      ├── Scalping 9 Robots
│      ├── XAUUSD H4 Solo Compras
│      └── EURUSD Mean Reversion
│
├── Strategies
│
├── Experiments
│
├── Backtests
│
├── Runs
│
├── Metrics
│
├── Decisions
│
└── Artifacts
```

---

7. ¿Por qué una DB central es mejor?

Principalmente porque **querés comparar conocimiento entre proyectos**.

Imaginemos que en un futuro tenés:

```
20 proyectos
500 estrategias
15.000 experimentos
```

Podrías preguntarle al sistema:

> *¿Qué tipo de estrategias tuvo mejor rendimiento out-of-sample?*

No importa en qué proyecto estaban.

O:

> *¿Qué estrategias sobre XAUUSD tuvieron drawdown menor al 10%?*

O:

> *¿Qué metodología produjo mayor porcentaje de estrategias que sobrevivieron Monte Carlo?*

Eso es prácticamente imposible de hacer elegantemente si cada proyecto tiene una base de datos separada.

Con una base central:

```
Projects
    ↓
Strategies
    ↓
Experiments
    ↓
Results
```

podés hacer análisis transversal.

---

8. Pero cada proyecto sigue teniendo su identidad

Eso no significa que todo quede mezclado.

Cada proyecto tendría algo como:

```
Projects/
└── Scalping 9 Robots/
    ├── README.md
    ├── MT5/
    ├── SQX/
    ├── Documentation/
    └── Session Reports/
```

y en la base:

```
project_id = 001
```

Entonces todas las entidades asociadas quedan relacionadas:

```
Project
   │
   ├── Strategy
   │       │
   │       └── Strategy Version
   │               │
   │               └── Experiment
   │                       │
   │                       ├── Backtest
   │                       ├── Metrics
   │                       └── Artifacts
   │
```

---

9. El esquema inicial de la base podría ser algo así

No lo implementaría todavía, pero conceptualmente:

```
projects
---------
id
name
slug
description
status
created_at
updated_at

strategies
----------
id
project_id
name
description
strategy_family
status
created_at

strategy_versions
-----------------
id
strategy_id
version
git_commit
configuration_hash
parent_version_id
created_at

experiments
-----------
id
project_id
strategy_version_id
type
hypothesis
status
started_at
completed_at
```

```
runs
----
id
experiment_id
engine
symbol
timeframe
start_date
end_date
data_version
configuration
started_at
completed_at

metrics
-------
id
run_id
metric_name
metric_value
```

Y después podríamos tener:

```
artifacts
---------
id
experiment_id
type
path
sha256
created_at
```

Esto último es interesante porque el archivo puede vivir físicamente en:

```
Projects/Scalping 9 Robots/MT5/...
```

mientras que la base guarda:

```
artifact_id
experiment_id
path
hash
tipo
```

---

10. Y no metería todo dentro de PostgreSQL

Acá también conviene separar responsabilidades.

Yo pensaría:

```
                TRADING LAB
                     |
     ┌───────────────┼───────────────┐
     |               |               |
   Git          Filesystem     PostgreSQL
     |               |               |
     |               |               |
código          reportes         experimentos
docs            archivos         métricas
AGENTS          artefactos       resultados
configs         documentos       relaciones
```

Git

Versionado.

Filesystem

Archivos reales.

PostgreSQL

Información estructurada.

Por ejemplo, un reporte MT5 de 4 MB **no necesita vivir como un blob dentro de PostgreSQL**.

Puede vivir en:

```
Projects/.../MT5/
```

y PostgreSQL guarda:

```
Experiment #421
↓
MT5 report
↓
Projects/.../MT5/report.html
↓
SHA256 = ...
```

---

11. ¿Dónde pondría la base de datos físicamente?

No:

```
Programming/Database/trading_lab.db
```

Si usamos PostgreSQL.

`Programming/Database/` tendría:

```
Programming/
└── Database/
    ├── Schema/
    ├── Migrations/
    ├── Queries/
    ├── Seeds/
    └── README.md
```

La base de datos sería un **servicio externo al árbol de archivos**.

Por ejemplo:

```
PostgreSQL
    ↑
Programming/Database/
    ↑
Applications / scripts / agents
```

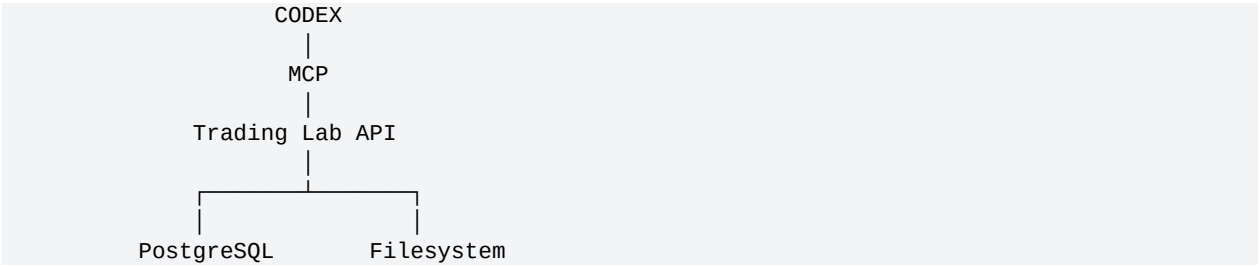
Y Codex puede interactuar con ella mediante scripts, CLI, MCP o herramientas específicas, sin que el agente tenga que manipular físicamente los archivos internos de PostgreSQL.

Esto además encaja muy bien con el concepto de harness: el agente tiene herramientas controladas para operar sobre la DB en vez de inventarse mecanismos ad hoc.

---

12. Incluso podemos tener una capa MCP

Más adelante podría quedar:



Entonces Codex podría recibir tareas como:

> *"Analizá las 100 estrategias del proyecto X y encontrá las que sobrevivieron los tests de robustez."*

El agente no debería recorrer manualmente miles de archivos.

Consulta:

```
MCP
 ↓
Database
 ↓
Resultados
 ↓
Codex
```

Y para algo como:

> *"Leé el último reporte MT5 de la estrategia 023"*

puede resolver:

```
DB
 ↓
artifact path
 ↓
archivo
```

---

13. ¿Y las sesiones?

Acá también mantendría una separación muy estricta.

Supongamos que Codex trabaja hoy:

```
Experiment #421
```

y hace tres cosas:

1. modifica un parser;
2. ejecuta tests;
3. analiza los resultados.

Al terminar:

Git

Guarda:

```
code changes
config changes
documentation changes
session report
```

PostgreSQL

Guarda:

```
experiment
run
metrics
artifact references
decision
```

Session Report

Guarda:

```
qué hicimos
qué descubrimos
qué decidimos
qué falta hacer
```

Ese modelo me parece **muy robusto para el tipo de laboratorio que estás imaginando**.

---

14. Y el harness de Codex

Ahí sí creo que tenemos que ser disciplinados.

En el root:

```
AGENTS.md
```

debería contener las reglas **globales** del laboratorio.

Luego:

```
Projects/X/AGENTS.md
```

solamente cuando existan reglas específicas de ese proyecto.

Y:

```
.agents/skills/
```

para workflows repetibles.

Por ejemplo:

```
.agents/
└── skills/
    ├── create-session-report/
    ├── analyze-mt5-report/
    ├── process-sqx-results/
    ├── register-experiment/
    ├── run-database-migration/
    └── validate-strategy/
```

Esto no es una idea arbitraria: OpenAI está usando precisamente `AGENTS.md` para reglas del repositorio y `.agents/skills/` para workflows reutilizables, y señala que las skills sirven para encapsular procedimientos repetibles. (https://developers.openai.com/blog/skills-agents-sdk?trk=article-ssr-frontend-pulse_little-text-block&utm_source=chatgpt.com)

Y la parte interesante para vos es que la propia documentación de OpenAI contempla la migración de un setup de Claude: `CLAUDE.md` de repositorio → `AGENTS.md`, y `.claude/skills` → `.agents/skills`. (https://developers.openai.com/cookbook/examples/agents_sdk/migrate-from-claude-agent-sdk/readme?utm_source=chatgpt.com)

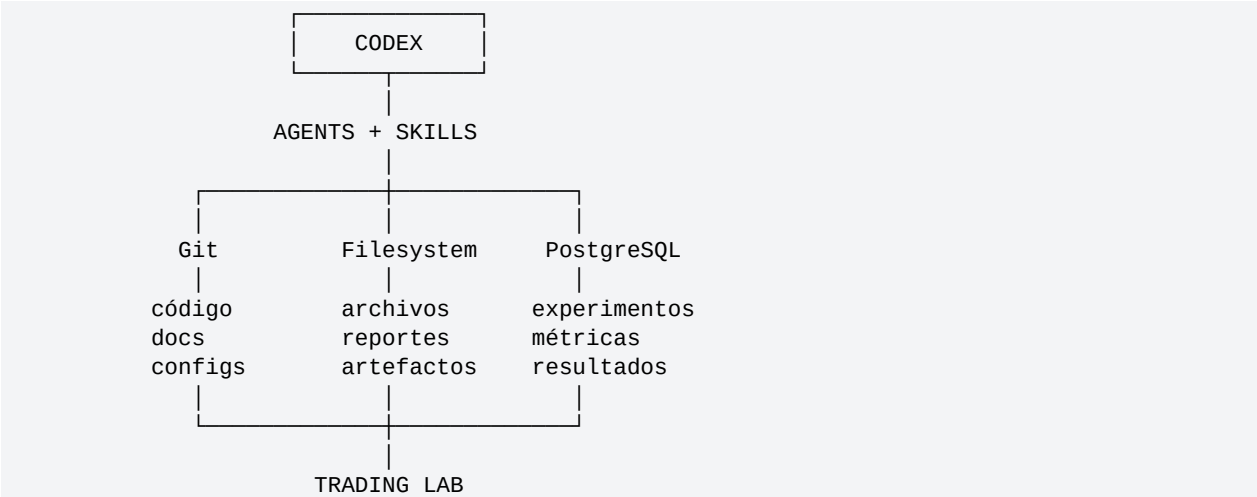
---

15. Mi arquitectura final

Entonces, después de revisar todo, **ésta sería mi versión recomendada**:

```
TRADING LAB
│
├── README.md
├── AGENTS.md                    ← reglas globales
├── .gitignore
│
├── .agents/
│   └── skills/                  ← workflows del agente
│
├── Session Reports/             ← memoria cronológica
│
├── Documentation/               ← conocimiento
│
├── Programming/                 ← infraestructura
│   ├── Applications/
│   ├── Automation/
│   ├── Scripts/
│   ├── Database/
│   │   ├── Schema/
│   │   ├── Migrations/
│   │   ├── Queries/
│   │   └── Seeds/
│   └── Integrations/
│
└── Projects/                    ← experimentación
    │
    ├── Project A/
    │   ├── README.md
    │   ├── AGENTS.md
    │   ├── MT5/
    │   ├── SQX/
    │   ├── Documentation/
    │   └── Session Reports/
    │
    └── Project B/
        ├── README.md
        ├── AGENTS.md
        ├── MT5/
        ├── SQX/
        ├── Documentation/
        └── Session Reports/
```

Y por fuera:



La decisión que más defendería es ésta: una sola base de datos para todo Trading Lab, un solo repositorio Git para todo Trading Lab y múltiples proyectos lógicos dentro de ambos.

Eso nos permite comparar proyectos, conservar la trazabilidad histórica y, sobre todo, hacer que Codex pueda trabajar sobre un sistema coherente en lugar de tener que entender una colección de proyectos aislados.

Y hay una consecuencia muy buena: **el proyecto no queda atado a Codex**. Codex es el agente que trabaja sobre el laboratorio; Git, PostgreSQL, los documentos y los proyectos constituyen el laboratorio en sí.

El siguiente paso que haría es diseñar **la estructura exacta de la base de datos (tablas, relaciones y qué información viene de SQX, qué viene de MT5 y qué información introduce el agente)** antes de escribir el `AGENTS.md`. Ahí vamos a definir realmente el "modelo mental" del Trading Lab.